

SOFIA.py — Sistema de Tutoría Socrática

Repositorio de casos de prueba · Universidad Pedagógica y Tecnológica de Colombia

Este documento contiene todos los ejercicios de práctica del sistema SOFIA organizados por concepto, con su descripción, salida esperada, solución de referencia y errores frecuentes. Cada ejercicio corresponde a un caso de prueba en el repositorio del evaluador (Capa 3).

Concepto	Ejercicios	Dificultades
Ciclos	3	Intermedio, Básico
Funciones	3	Intermedio, Básico
Listas	3	Intermedio, Básico
Strings	3	Intermedio, Básico
Condicionales	2	Básico
Diccionarios	2	Intermedio, Básico
Recursión	2	Intermedio
Variables y Tipos	2	Básico

Ciclos

1. Imprimir del 1 al 5 [BÁSICO]

Enunciado:

Usa un ciclo for con range() para imprimir los números del 1 al 5, uno por línea.

Salida esperada:

```
1 / 2 / 3 / 4 / 5
```

Solución de referencia:

```
for i in range(1, 6):  
    print(i)
```

Errores frecuentes:

Usar range(1, 5) omite el 5. Usar range(6) empieza desde 0.

2. Suma del 1 al 10 [BÁSICO]

Enunciado:

Usa un ciclo para calcular la suma acumulada del 1 al 10 e imprime el resultado.

Salida esperada:

```
55
```

Solución de referencia:

```
total = 0  
for i in range(1, 11):  
    total += i  
print(total)
```

Errores frecuentes:

No inicializar total en 0. Imprimir dentro del ciclo en vez de afuera.

3. Tabla del 3 [INTERMEDIO]

Enunciado:

Imprime los primeros 10 múltiplos de 3 (del 3 al 30), uno por línea.

Salida esperada:

```
3 / 6 / 9 / 12 / 15 / 18 / 21 / 24 / 27 / 30
```

Solución de referencia:

```
for i in range(1, 11):  
    print(3 * i)
```

Errores frecuentes:

Usar range(3, 31) con paso 3 también funciona pero es menos claro.

Funciones

4. Función doble [BÁSICO]

Enunciado:

Escribe una función `doble(x)` que retorne el doble de `x`. Imprime `doble(5)`.

Salida esperada:

10

Solución de referencia:

```
def doble(x):  
    return x * 2  
print(doble(5))
```

Errores frecuentes:

Usar `print()` dentro de la función en vez de `return`. No llamar la función.

5. ¿Es par? [BÁSICO]

Enunciado:

Escribe `es_par(n)` que retorne `True` si `n` es par. Imprime `es_par(4)` y `es_par(3)`.

Salida esperada:

True / False

Solución de referencia:

```
def es_par(n):  
    return n % 2 == 0  
print(es_par(4))  
print(es_par(3))
```

Errores frecuentes:

Confundir `%` con `//`. Retornar string `'True'` en vez de booleano `True`.

6. Factorial recursivo [INTERMEDIO]

Enunciado:

Escribe `factorial(n)` recursiva. Imprime `factorial(5)`.

Salida esperada:

120

Solución de referencia:

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)  
print(factorial(5))
```

Errores frecuentes:

Olvidar el caso base. Caso base con `n==1` falla para `n==0`.

Listas

7. Suma de lista [BÁSICO]

Enunciado:

Dada [1,2,3,4,5], calcula e imprime la suma sin usar sum().

Salida esperada:

15

Solución de referencia:

```
numeros = [1, 2, 3, 4, 5]
total = 0
for n in numeros:
    total += n
print(total)
```

Errores frecuentes:

Imprimir dentro del ciclo. Olvidar inicializar total.

8. Lista invertida [INTERMEDIO]

Enunciado:

Dada [1,2,3,4,5], imprímela invertida: [5,4,3,2,1].

Salida esperada:

[5, 4, 3, 2, 1]

Solución de referencia:

```
numeros = [1, 2, 3, 4, 5]
print(numeros[::-1])
```

Errores frecuentes:

Usar reverse() modifica la lista original. El slicing[::-1] no la modifica.

9. Filtrar pares [INTERMEDIO]

Enunciado:

Dada [1,2,3,4,5,6], imprime solo los pares: [2,4,6].

Salida esperada:

[2, 4, 6]

Solución de referencia:

```
numeros = [1, 2, 3, 4, 5, 6]
print([x for x in numeros if x % 2 == 0])
```

Errores frecuentes:

Imprimir elemento por elemento en vez de la lista completa.

Strings

10. Invertir string [BÁSICO]

Enunciado:

Invierte 'Python' e imprímelo. Resultado: nohtyP

Salida esperada:

nohtyP

Solución de referencia:

```
s = 'Python'
print(s[::-1])
```

Errores frecuentes:

Usar `reversed()` retorna un iterador, no un string directamente.

11. Contar vocales [BÁSICO]

Enunciado:

Cuenta e imprime las vocales de 'murciélago'. Resultado: 5

Salida esperada:

5

Solución de referencia:

```
palabra = 'murciélago'
print(sum(1 for c in palabra if c in 'aeiouáéíóú'))
```

Errores frecuentes:

Olvidar las vocales con tilde. No manejar mayúsculas/minúsculas.

12. Palíndromo [INTERMEDIO]

Enunciado:

Verifica si 'ana' es palíndromo (True) y 'luis' (False). Imprime ambos.

Salida esperada:

True / False

Solución de referencia:

```
def palindromo(s):
    return s == s[::-1]
print(palindromo('ana'))
print(palindromo('luis'))
```

Errores frecuentes:

Comparar con `lower()` cuando no es necesario. Invertir con loop.

Condicionales

13. Positivo, negativo o cero [BÁSICO]

Enunciado:

Dado x=7, imprime 'positivo', 'negativo' o 'cero'.

Salida esperada:

positivo

Solución de referencia:

```
x = 7
if x > 0:
    print('positivo')
elif x < 0:
    print('negativo')
else:
    print('cero')
```

Errores frecuentes:

Usar = en vez de ==. Olvidar los dos puntos al final del if.

14. Mayor de edad [BÁSICO]

Enunciado:

Dado edad=20, imprime 'mayor de edad' si ≥ 18 , si no 'menor de edad'.

Salida esperada:

mayor de edad

Solución de referencia:

```
edad = 20
if edad >= 18:
    print('mayor de edad')
else:
    print('menor de edad')
```

Errores frecuentes:

Usar > en vez de $\geq$. Escribir el string diferente al esperado.

Diccionarios

15. Acceder a clave [BÁSICO]

Enunciado:

Crea un diccionario con clave 'lenguaje' y valor 'Python'. Imprime el valor.

Salida esperada:

Python

Solución de referencia:

```
d = {'lenguaje': 'Python'}  
print(d['lenguaje'])
```

Errores frecuentes:

Usar paréntesis en vez de corchetes para acceder. `KeyError` si la clave no existe.

16. Frecuencia de letras [INTERMEDIO]

Enunciado:

Cuenta cuántas veces aparece cada letra en 'hola' e imprime el dict.

Salida esperada:

```
{'h': 1, 'o': 1, 'l': 1, 'a': 1}
```

Solución de referencia:

```
palabra = 'hola'  
freq = {}  
for c in palabra:  
    freq[c] = freq.get(c, 0) + 1  
print(freq)
```

Errores frecuentes:

`KeyError` si no se inicializa la clave. Usar `[]` en vez de `get()`.

Recursión

17. Fibonacci recursivo [INTERMEDIO]

Enunciado:

Escribe fibonacci(n) recursiva e imprime fibonacci(6). Resultado: 8

Salida esperada:

8

Solución de referencia:

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)  
print(fibonacci(6))
```

Errores frecuentes:

Caso base incorrecto. Sumar n-1 y n-2 en vez de fibonacci de esos valores.

18. Suma recursiva [INTERMEDIO]

Enunciado:

Escribe suma_recursiva(n) que sume del 1 al n. Imprime suma_recursiva(4). Resultado: 10

Salida esperada:

10

Solución de referencia:

```
def suma_recursiva(n):  
    if n == 0:  
        return 0  
    return n + suma_recursiva(n-1)  
print(suma_recursiva(4))
```

Errores frecuentes:

Caso base n==1 retorna 1 pero falla para n==0.

Variables y Tipos

19. Imprimir entero [BÁSICO]

Enunciado:

Declara una variable entera con valor 42 e imprímela.

Salida esperada:

```
42
```

Solución de referencia:

```
x = 42  
print(x)
```

Errores frecuentes:

Imprimir '42' (string) en vez de 42 (int).

20. Conversión de tipos [BÁSICO]

Enunciado:

Convierte el string '15' a entero, súmale 10 e imprime el resultado. Resultado: 25

Salida esperada:

```
25
```

Solución de referencia:

```
texto = '15'  
print(int(texto) + 10)
```

Errores frecuentes:

Sumar sin convertir da '1510' (concatenación). TypeError si se olvida int().
